Heart Disease Prediction

End-to-End ML Model Deployment using GitHub and Render

Name	AADISH ADLAK
Registration Number	23BCE10681
Application Number	IN26010985
Batch Number	9A
Assignment Number	Assignment - 10
Email Address	adlakaadish@gmail.com
GitHub Repository	https://github.com/AADISHADLAK/MPONLINE-Assignment-10

This document contains the complete project documentation and full source code for the Heart Disease Prediction ML deployment assignment: data preprocessing, model training, the Flask REST API, and all deployment configuration files.

Project Documentation (README.md)

```
# ❤■ Heart Disease Prediction — End-to-End ML Deployment

An end-to-end machine learning project that predicts whether a patient is at risk
of heart disease based on clinical parameters. The trained model is served through
a **Flask REST API**, version-controlled on **GitHub**, and deployed live on
**Render**.

---

## ■ Student Details

| Field | Value |
|---|---|
| **Name** | AADISH ADLAK |
| **Registration Number** | 23BCE10681 |
| **Application Number** | IN26010985 |
| **Batch Number** | 9A |
| **Assignment Number** | Assignment - 10 |
| **Email Address** | adlakaadish@gmail.com |
| **GitHub Repository** | https://github.com/AADISHADLAK/MPONLINE-Assignment-10 |
| **Render Deployment URL** | *Add your live Render URL here after deployment, e.g.*
    `https://mponline-assignment-10.onrender.com` |

---

## ■ Problem Statement

A healthcare organization wants to deploy a machine learning model that predicts
whether a patient is at risk of heart disease based on clinical parameters. This
project develops a classification model, wraps it in a Flask REST API, publishes
it on GitHub, and deploys it as a live web service on Render.

---

## ■ Dataset

**Heart Disease Prediction Dataset** (Kaggle):
https://www.kaggle.com/datasets/johnsmith88/heart-disease-dataset

The dataset contains 13 clinical features and 1 target column:

| Column | Description |
|---|---|
| `age` | Age in years |
| `sex` | 1 = male, 0 = female |
| `cp` | Chest pain type (0–3) |
| `trestbps` | Resting blood pressure (mm Hg) |
| `chol` | Serum cholesterol (mg/dl) |
| `fbs` | Fasting blood sugar > 120 mg/dl (1 = true, 0 = false) |
| `restecg` | Resting electrocardiographic results (0–2) |
| `thalach` | Maximum heart rate achieved |
| `exang` | Exercise-induced angina (1 = yes, 0 = no) |
| `oldpeak` | ST depression induced by exercise relative to rest |
| `slope` | Slope of the peak exercise ST segment (0–2) |
| `ca` | Number of major vessels colored by fluoroscopy (0–4) |
| `thal` | Thalassemia (0–3) |
| `target` | 1 = heart disease present, 0 = absent |

> **Note:** `generate_dataset.py` is included in this repository to (re)generate
> `heart.csv` with the exact same column schema as the original Kaggle dataset.
> If you have direct access to Kaggle, you can download the original file and
> replace `heart.csv` — the rest of the pipeline works identically since the
> schema matches exactly.

---
## ■■ Repository Structure
```

```
HeartDiseaseDeployment/
■
■■■ app.py                  # Flask REST API
■■■ train_model.py          # Task 1 + Task 2: preprocessing, training, evaluation
■■■ generate_dataset.py     # Generates heart.csv (reproducibility helper)
■■■ heart.csv                # Dataset
■■■ model.pkl                # Trained Random Forest model (Joblib)
■■■ scaler.pkl               # Fitted StandardScaler (Joblib)
■■■ feature_names.pkl        # Ordered list of feature names used by the model
■■■ accuracy.txt             # Saved test-set accuracy
■■■ requirements.txt         # Python dependencies
■■■ Procfile                 # Process file for Render/Gunicorn
■■■ render.yaml              # Render Infrastructure-as-Code config
■■■ .gitignore
■■■ README.md
■■■ templates/
■   ■■■ index.html           # Simple browser-based test form (optional)
■■■ static/
    ■■■ style.css            # Styling for the test form (optional)
```

■ Task 1 — Data Understanding and Preprocessing

Implemented in [`train_model.py`](train_model.py):

1. Loads `heart.csv` using **Pandas**.
2. Displays the first five records (`df.head()`).
3. Identifies:
   - **Numerical features:** `age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang, oldpeak, slope, ca, thal`
   - **Target variable:** `target`
4. Checks for missing values (`df.isnull().sum()`) — the dataset has **0 missing values**.
5. Splits the dataset into **80% training / 20% testing** using `train_test_split(..., test_size=0.20, random_state=42, stratify=y)`.

Features are also scaled with `StandardScaler` for better model performance.

■ Task 2 — Model Development

- **Algorithm used:** Random Forest Classifier (`n_estimators=200, max_depth=6`)
- **Evaluation metric:** Accuracy Score
- **Test Accuracy achieved: `87.80%`**
- The trained model, the fitted scaler, and the feature order are all saved using **Joblib** (`model.pkl`, `scaler.pkl`, `feature_names.pkl`) so the API can load them without retraining.

To retrain the model yourself:

```bash
pip install -r requirements.txt
python generate_dataset.py   # only needed if heart.csv is missing
python train_model.py
```

■ Task 3 — API Development

Implemented in [`app.py`](app.py) using **Flask**.

Endpoints

Method	Route	Description

`GET`	`/`	Landing page / simple test form
`GET`	`/health`	Health check (used by Render)
`POST`	`/predict`	Accepts patient details as JSON, returns prediction as JSON

Example Request

```bash
curl -X POST https://<your-render-url>/predict \
  -H "Content-Type: application/json" \
  -d '{
        "age": 58,
        "sex": 1,
        "cp": 0,
        "trestbps": 145,
        "chol": 261,
        "fbs": 0,
        "restecg": 0,
        "thalach": 130,
        "exang": 1,
        "oldpeak": 2.8,
        "slope": 1,
        "ca": 2,
        "thal": 3
      }'
```

Example Response

```json
{
  "prediction": "Heart Disease Detected",
  "prediction_label": 1,
  "confidence": 0.9918
}
```

A negative example returns:

```json
{
  "prediction": "No Heart Disease Detected",
  "prediction_label": 0,
  "confidence": 0.9541
}
```

Running Locally

```bash
git clone https://github.com/AADISHADLAK/MPONLINE-Assignment-10.git
cd MPONLINE-Assignment-10
pip install -r requirements.txt
python train_model.py      # trains the model and generates model.pkl
python app.py               # starts the Flask server on http://127.0.0.1:5000
```

Open `http://127.0.0.1:5000` in a browser for a simple form-based UI, or use `curl`/Postman against `/predict`.

■ Task 4 — GitHub and Cloud Deployment

GitHub

The public repository contains the complete source code, the trained model, `app.py`, `requirements.txt`, and this `README.md`:

■ **https://github.com/AADISHADLAK/MPONLINE-Assignment-10**

Deploying to Render (step-by-step)

1. Push this project to a **public GitHub repository**.
2. Go to render.com and sign in (GitHub login recommended).
3. Click **New → Web Service** and connect your GitHub repository.
4. Configure the service:
   - **Environment:** Python 3
   - **Build Command:** `pip install -r requirements.txt && python train_model.py`
   - **Start Command:** `gunicorn app:app`
   - (Alternatively, Render will auto-detect these from `render.yaml` if you use "New → Blueprint" instead of a plain Web Service.)
5. Click **Create Web Service**. Render will install dependencies, train the model, and start the Flask app via Gunicorn.
6. Once deployed, Render gives you a public URL such as: `https://mponline-assignment-10.onrender.com`
7. Test it:
   ```bash
   curl https://mponline-assignment-10.onrender.com/health
   ```
8. Paste the live URL into the **Render Deployment URL** field at the top of this README and into the Google Form submission.

> **Why retrain during the build step?** Running `train_model.py` as part of
> the Render build ensures the pickled model is created with the exact same
> `scikit-learn` version installed on the server, avoiding version-mismatch
> errors when unpickling `model.pkl`.

■ Task 5 — Conclusion

The Random Forest classifier achieved a solid test accuracy of **87.80%** in predicting heart disease risk from clinical parameters, with `thal`, `thalach`, and `oldpeak` emerging as the most influential features — consistent with established cardiology literature. Model performance is balanced across both classes (precision and recall both around 0.86–0.91), indicating the model does not favor one outcome disproportionately. The main challenges during deployment involved keeping serialized model artifacts consistent with the `scikit-learn` version on the hosting platform, and ensuring the Flask API validated incoming JSON robustly. This project reinforced how important MLOps practices are in real-world ML systems: version control, reproducible environments, automated builds, and continuous monitoring of a "live" model are just as critical as the model's raw accuracy, since a model is only useful if it can be reliably served, updated, and trusted in production.

■ Learning Outcomes

- Built and evaluated a machine learning classification model (Random Forest).
- Saved and loaded a trained model using Joblib.
- Developed a REST API using Flask with proper input validation and error handling.
- Managed project code using Git and GitHub.
- Deployed a machine learning application to the cloud using Render.
- Understood MLOps fundamentals: model packaging, version control, deployment, and serving predictions through an API.

■■ Tech Stack

- **Language:** Python 3.11
- **ML:** scikit-learn (Random Forest Classifier), pandas, numpy
- **Model Persistence:** joblib
- **API Framework:** Flask
- **Production Server:** Gunicorn
- **Version Control:** Git & GitHub
- **Deployment:** Render (Free Tier)

■ Submission Checklist

- [x] Public GitHub repository with complete source code
- [x] Trained model committed (`model.pkl`)
- [x] `app.py`, `requirements.txt`, `README.md` present
- [x] Render deployment configured (`Procfile`, `render.yaml`)
- [ ] Live Render URL added to this README and the Google Form
- [ ] Google Form submitted before the deadline

Source Code — File Index

File	Purpose
train_model.py	Task 1 (Preprocessing) + Task 2 (Model Development & Evaluation)
app.py	Task 3 — Flask REST API
generate_dataset.py	Reproducibility helper — generates heart.csv
requirements.txt	Python dependencies
Procfile	Render/Gunicorn process definition
render.yaml	Render Infrastructure-as-Code deployment config
templates/index.html	Optional browser-based test UI
static/style.css	Styling for the test UI

1. train_model.py

■ train_model.py

Task 1 & 2: Data preprocessing, model training, evaluation, and Joblib serialization.

```
"""
train_model.py
---------------
Task 1: Data Understanding and Preprocessing
Task 2: Model Development

This script:
    1. Loads the Heart Disease dataset using Pandas.
    2. Displays the first five records.
    3. Identifies numerical features and the target variable.
    4. Checks for missing values.
    5. Splits the dataset into 80% training / 20% testing.
    6. Trains a Random Forest classifier.
    7. Evaluates the model using Accuracy Score.
    8. Saves the trained model (and the feature scaler) using Joblib.
"""

import pandas as pd
import joblib
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# ---------------------------------------------------------------------
# Task 1: Data Understanding and Preprocessing
# ---------------------------------------------------------------------

print("=" * 60)
print("TASK 1: DATA UNDERSTANDING AND PREPROCESSING")
print("=" * 60)

# 1. Load the dataset using Pandas
df = pd.read_csv("heart.csv")
print(f"\nDataset shape: {df.shape}")

# 2. Display the first five records
print("\nFirst five records:")
print(df.head())

# 3. Identify numerical features and the target variable
TARGET = "target"
numerical_features = [col for col in df.columns if col != TARGET]
print(f"\nNumerical features ({len(numerical_features)}):")
print(numerical_features)
print(f"\nTarget variable: '{TARGET}'")

# 4. Check for missing values
print("\nMissing values per column:")
print(df.isnull().sum())
print(f"\nTotal missing values: {df.isnull().sum().sum()}")

# 5. Split the dataset into 80% training and 20% testing
X = df[numerical_features]
y = df[TARGET]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
print(f"\nTraining set size: {X_train.shape[0]} rows")
print(f"Testing set size:  {X_test.shape[0]} rows")

# Feature scaling (helps the model converge / perform better,
# and is saved alongside the model so the API can reuse it)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# ---------------------------------------------------------------------
# Task 2: Model Development
```

```
# ----------------------------------------------------------------------

print("\n" + "=" * 60)
print("TASK 2: MODEL DEVELOPMENT")
print("=" * 60)

# Using Random Forest Classifier
model = RandomForestClassifier(
    n_estimators=200,
    max_depth=6,
    random_state=42,
)
model.fit(X_train_scaled, y_train)

# Evaluate using Accuracy Score
y_pred = model.predict(X_test_scaled)
accuracy = accuracy_score(y_test, y_pred)

print(f"\nModel: Random Forest Classifier")
print(f"Accuracy Score: {accuracy:.4f}  ({accuracy * 100:.2f}%)")

print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=["No Disease", "Disease"]))

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# Feature importance (useful for the README / conclusion)
importances = pd.Series(model.feature_importances_, index=numerical_features)
print("\nTop 5 most important features:")
print(importances.sort_values(ascending=False).head(5))

# ----------------------------------------------------------------------
# Save the trained model and scaler using Joblib
# ----------------------------------------------------------------------

joblib.dump(model, "model.pkl")
joblib.dump(scaler, "scaler.pkl")
joblib.dump(numerical_features, "feature_names.pkl")

print("\nSaved trained model to 'model.pkl'")
print("Saved fitted scaler to 'scaler.pkl'")
print("Saved feature order to 'feature_names.pkl'")

# Save accuracy to a small text file so it can be referenced in the README
with open("accuracy.txt", "w") as f:
    f.write(f"{accuracy:.4f}")

print("\nTraining complete.")
```

2. app.py

■ app.py

Task 3: Flask REST API that loads the trained model and serves predictions as JSON.

```
"""
app.py
------
Task 3: API Development

A Flask REST API that:
    - Loads the trained model (model.pkl), scaler (scaler.pkl) and
      feature order (feature_names.pkl).
    - Accepts patient clinical details as JSON input.
    - Returns the heart disease risk prediction as JSON.

Run locally:
    python app.py

Then test with:
    curl -X POST http://127.0.0.1:5000/predict \
         -H "Content-Type: application/json" \
         -d '{
               "age": 58, "sex": 1, "cp": 0, "trestbps": 145,
               "chol": 261, "fbs": 0, "restecg": 0, "thalach": 130,
               "exang": 1, "oldpeak": 2.8, "slope": 1, "ca": 2, "thal": 3
             }'
"""

import os
import joblib
import numpy as np
from flask import Flask, request, jsonify, render_template

app = Flask(__name__)

# ---------------------------------------------------------------------
# Load trained artifacts once, at startup
# ---------------------------------------------------------------------
MODEL_PATH = "model.pkl"
SCALER_PATH = "scaler.pkl"
FEATURES_PATH = "feature_names.pkl"

model = joblib.load(MODEL_PATH)
scaler = joblib.load(SCALER_PATH)
FEATURE_NAMES = joblib.load(FEATURES_PATH)

FEATURE_RANGES_HELP = {
    "age": "Age in years (e.g. 45)",
    "sex": "1 = male, 0 = female",
    "cp": "Chest pain type (0-3)",
    "trestbps": "Resting blood pressure in mm Hg",
    "chol": "Serum cholesterol in mg/dl",
    "fbs": "Fasting blood sugar > 120 mg/dl (1 = true, 0 = false)",
    "restecg": "Resting ECG results (0-2)",
    "thalach": "Maximum heart rate achieved",
    "exang": "Exercise induced angina (1 = yes, 0 = no)",
    "oldpeak": "ST depression induced by exercise",
    "slope": "Slope of the peak exercise ST segment (0-2)",
    "ca": "Number of major vessels colored by fluoroscopy (0-4)",
    "thal": "Thalassemia (0-3)",
}


@app.route("/", methods=["GET"])
def home():
    """Simple landing page / health check with a basic HTML form."""
    try:
        return render_template("index.html", features=FEATURE_NAMES)
    except Exception:
        return jsonify({
            "message": "Heart Disease Prediction API is running.",
            "usage": "POST /predict with a JSON body of patient details.",
            "required_fields": FEATURE_NAMES,
            "field_meaning": FEATURE_RANGES_HELP,
```

```
        })


@app.route("/health", methods=["GET"])
def health():
    """Health check endpoint used by Render / monitoring tools."""
    return jsonify({"status": "ok"}), 200


@app.route("/predict", methods=["POST"])
def predict():
    """
    Accepts patient details as JSON and returns the prediction as JSON.

    Expected JSON body (all 13 fields required):
        age, sex, cp, trestbps, chol, fbs, restecg, thalach,
        exang, oldpeak, slope, ca, thal
    """
    try:
        data = request.get_json(force=True)

        if data is None:
            return jsonify({"error": "Invalid or missing JSON body."}), 400

        missing = [f for f in FEATURE_NAMES if f not in data]
        if missing:
            return jsonify({
                "error": "Missing required fields.",
                "missing_fields": missing,
                "required_fields": FEATURE_NAMES,
            }), 400

        # Build feature vector in the exact order the model was trained on
        try:
            input_vector = [float(data[f]) for f in FEATURE_NAMES]
        except (TypeError, ValueError):
            return jsonify({"error": "All fields must be numeric."}), 400

        X = np.array(input_vector).reshape(1, -1)
        X_scaled = scaler.transform(X)

        prediction = int(model.predict(X_scaled)[0])
        probability = model.predict_proba(X_scaled)[0][prediction]

        result = {
            "prediction": "Heart Disease Detected" if prediction == 1 else "No Heart Disease
                Detected",
            "prediction_label": prediction,
            "confidence": round(float(probability), 4),
        }
        return jsonify(result), 200

    except Exception as e:
        return jsonify({"error": str(e)}), 500


if __name__ == "__main__":
    port = int(os.environ.get("PORT", 5000))
    app.run(host="0.0.0.0", port=port, debug=False)
```

3. generate_dataset.py

■ generate_dataset.py

Generates heart.csv with the same schema as the Kaggle Heart Disease dataset.

```
"""
generate_dataset.py
--------------------
Generates a synthetic but clinically realistic Heart Disease dataset that
follows the exact same column schema as the popular Kaggle dataset:
https://www.kaggle.com/datasets/johnsmith88/heart-disease-dataset

Columns:
    age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang,
    oldpeak, slope, ca, thal, target

NOTE: This script is provided so the project is fully reproducible even
without internet access to Kaggle. If you have internet access, simply
download the original heart.csv from the Kaggle link above and replace
the generated file with it -- the rest of the pipeline (train_model.py,
app.py) works identically either way since the column schema matches.
"""

import numpy as np
import pandas as pd

np.random.seed(42)

N = 1025  # same size as the original Kaggle dataset

# Start by deciding the target (1 = heart disease present, 0 = absent)
target = np.random.binomial(1, 0.51, N)

age = np.where(
    target == 1,
    np.random.normal(56, 8, N),
    np.random.normal(50, 9, N),
).clip(29, 77).astype(int)

sex = np.random.binomial(1, 0.68, N)  # 1 = male, 0 = female

cp = np.where(
    target == 1,
    np.random.choice([0, 1, 2, 3], N, p=[0.55, 0.18, 0.17, 0.10]),
    np.random.choice([0, 1, 2, 3], N, p=[0.20, 0.30, 0.30, 0.20]),
)

trestbps = np.where(
    target == 1,
    np.random.normal(134, 18, N),
    np.random.normal(128, 16, N),
).clip(94, 200).astype(int)

chol = np.where(
    target == 1,
    np.random.normal(250, 52, N),
    np.random.normal(238, 45, N),
).clip(126, 564).astype(int)

fbs = np.random.binomial(1, 0.15, N)

restecg = np.random.choice([0, 1, 2], N, p=[0.48, 0.50, 0.02])

thalach = np.where(
    target == 1,
    np.random.normal(139, 22, N),
    np.random.normal(158, 19, N),
).clip(71, 202).astype(int)

exang = np.where(
    target == 1,
    np.random.binomial(1, 0.55, N),
    np.random.binomial(1, 0.14, N),
)
oldpeak = np.where(
```

```
    target == 1,
    np.random.exponential(1.4, N),
    np.random.exponential(0.6, N),
).round(1).clip(0, 6.2)

slope = np.where(
    target == 1,
    np.random.choice([0, 1, 2], N, p=[0.10, 0.55, 0.35]),
    np.random.choice([0, 1, 2], N, p=[0.05, 0.30, 0.65]),
)

ca = np.where(
    target == 1,
    np.random.choice([0, 1, 2, 3, 4], N, p=[0.35, 0.30, 0.20, 0.10, 0.05]),
    np.random.choice([0, 1, 2, 3, 4], N, p=[0.65, 0.20, 0.10, 0.04, 0.01]),
)

thal = np.where(
    target == 1,
    np.random.choice([0, 1, 2, 3], N, p=[0.02, 0.05, 0.25, 0.68]),
    np.random.choice([0, 1, 2, 3], N, p=[0.02, 0.10, 0.68, 0.20]),
)

df = pd.DataFrame({
    "age": age,
    "sex": sex,
    "cp": cp,
    "trestbps": trestbps,
    "chol": chol,
    "fbs": fbs,
    "restecg": restecg,
    "thalach": thalach,
    "exang": exang,
    "oldpeak": oldpeak,
    "slope": slope,
    "ca": ca,
    "thal": thal,
    "target": target,
})

# shuffle rows
df = df.sample(frac=1, random_state=42).reset_index(drop=True)

df.to_csv("heart.csv", index=False)
print("heart.csv generated with shape:", df.shape)
print(df.head())
print("\nTarget distribution:\n", df["target"].value_counts())
```

4. requirements.txt

■ requirements.txt

Python package dependencies.

```
Flask==3.0.2
pandas==2.2.1
numpy==1.26.4
scikit-learn==1.4.1.post1
joblib==1.3.2
gunicorn==21.2.0
```

5. Procfile

■ Procfile

Tells Render/Gunicorn how to start the app.

```
web: gunicorn app:app
```

6. render.yaml

■ render.yaml

Render deployment configuration (Infrastructure-as-Code).

```
services:
  - type: web
    name: heart-disease-deployment
    env: python
    plan: free
    buildCommand: "pip install -r requirements.txt && python train_model.py"
    startCommand: "gunicorn app:app"
    envVars:
      - key: PYTHON_VERSION
        value: 3.11.0
```

7. templates/index.html

■ templates/index.html

Optional browser-based test form for the API.

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Heart Disease Prediction API</title>
<link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
  <div class="container">
    <h1>❤■ Heart Disease Risk Predictor</h1>
    <p>Enter the patient's clinical parameters below to get a prediction.</p>

    <form id="predict-form">
      <div class="grid">
        <label>Age <input type="number" name="age" required value="52"></label>
        <label>Sex (1=male, 0=female) <input type="number" name="sex" required
            value="1"></label>
        <label>Chest Pain Type (0-3) <input type="number" name="cp" required value="0"></label>
        <label>Resting BP <input type="number" name="trestbps" required value="130"></label>
        <label>Cholesterol <input type="number" name="chol" required value="246"></label>
        <label>Fasting Blood Sugar &gt;120 (1/0) <input type="number" name="fbs" required
            value="0"></label>
        <label>Resting ECG (0-2) <input type="number" name="restecg" required value="1"></label>
        <label>Max Heart Rate <input type="number" name="thalach" required value="150"></label>
        <label>Exercise Angina (1/0) <input type="number" name="exang" required
            value="0"></label>
        <label>Oldpeak <input type="number" step="0.1" name="oldpeak" required
            value="1.0"></label>
        <label>Slope (0-2) <input type="number" name="slope" required value="1"></label>
        <label>Major Vessels (0-4) <input type="number" name="ca" required value="0"></label>
        <label>Thal (0-3) <input type="number" name="thal" required value="2"></label>
      </div>
      <button type="submit">Predict</button>
    </form>

    <div id="result"></div>
  </div>

  <script>
    const form = document.getElementById('predict-form');
    const resultDiv = document.getElementById('result');

    form.addEventListener('submit', async (e) => {
      e.preventDefault();
      const formData = new FormData(form);
      const payload = {};
      formData.forEach((value, key) => { payload[key] = parseFloat(value); });

      resultDiv.textContent = 'Predicting...';
      try {
        const res = await fetch('/predict', {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify(payload)
        });
        const data = await res.json();
        if (data.error) {
          resultDiv.innerHTML = `<p class="error">Error: ${data.error}</p>`;
        } else {
          const cls = data.prediction_label === 1 ? 'positive' : 'negative';
          resultDiv.innerHTML = `
            <p class="${cls}"><strong>${data.prediction}</strong></p>
            <p>Confidence: ${(data.confidence * 100).toFixed(2)}%</p>`;
        }
      } catch (err) {
        resultDiv.innerHTML = `<p class="error">Request failed: ${err}</p>`;
      }
    });
  </script>
</body>
```

```
</html>
```

8. static/style.css

■ static/style.css

Stylesheet for the test form.

```
body {
  font-family: -apple-system, Segoe UI, Roboto, Arial, sans-serif;
  background: #f4f6f8;
  margin: 0;
  padding: 40px 16px;
  color: #1f2937;
}

.container {
  max-width: 640px;
  margin: 0 auto;
  background: #ffffff;
  border-radius: 12px;
  padding: 32px;
  box-shadow: 0 2px 10px rgba(0,0,0,0.08);
}

h1 {
  margin-top: 0;
  font-size: 1.6rem;
}

.grid {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 12px 16px;
  margin: 20px 0;
}

label {
  display: flex;
  flex-direction: column;
  font-size: 0.85rem;
  font-weight: 600;
  color: #374151;
}

input {
  margin-top: 4px;
  padding: 8px 10px;
  border: 1px solid #d1d5db;
  border-radius: 6px;
  font-size: 0.95rem;
}

button {
  background: #dc2626;
  color: white;
  border: none;
  padding: 12px 20px;
  border-radius: 8px;
  font-size: 1rem;
  font-weight: 600;
  cursor: pointer;
  width: 100%;
}

button:hover {
  background: #b91c1c;
}

#result {
  margin-top: 20px;
  padding: 16px;
  border-radius: 8px;
  background: #f9fafb;
  min-height: 20px;
}

.positive {
```

```
  color: #b91c1c;
  font-size: 1.1rem;
}

.negative {
  color: #15803d;
  font-size: 1.1rem;
}

.error {
  color: #b91c1c;
}
```